Internship Mentorship Handbook

EduGuide AI - Intelligent Tutor

Project Name:	"EduGuide AI" - Intelligent Tutor
Assigned Student:	Disha Hareshkumar Rasdhari
Mentorship Focus:	Learn → Build → Integrate → Test → Document → Present

1. Project Overview

Problem Statement

Self-guided learning from textbook PDFs and digital courses is often passive. Students read paragraphs of complex text, but when they encounter confusing jargon or abstract concepts, they have no way to ask questions in context. Furthermore, passive reading does not measure comprehension. Human tutors provide personalized assessments and dialogic explanations, but they are expensive, unavailable 24/7, and do not scale.

Project Goal

The goal of this project is to build **EduGuide AI**, a web-based, RAG-powered (Retrieval-Augmented Generation) intelligent tutoring application using Python, Streamlit, and the Google Gemini API. The application will ingest a student's course textbook or lecture notes (PDF), index it locally in a vector database, allow the student to ask questions and receive context-informed explanations, and automatically generate interactive multi-choice quizzes directly derived from the uploaded materials to test understanding.

Real-world Applications

- **Adaptive Learning Portals:** Providing students with personalized study material reviews.
- **Corporate Training Assistants:** Ingesting employee handbooks or technical manuals to test and guide new hires.
- **Exam Preparation Tools:** Allowing students to upload syllabus sheets and study guides for custom quizzes.

Why This Project Matters

RAG is the industry standard pattern for building AI applications over private data. By building this project, interns master document parsing, text chunking strategies, vector embeddings generation, and vector index retrieval. Additionally, they learn how to enforce **structured JSON schemas** on LLM outputs to render dynamic frontend forms.

Expected Final MVP

A Streamlit web application that lets a student:

1. Upload a PDF textbook or syllabus.
2. Chat with the document in a conversational interface (RAG answers citing specific pages/paragraphs).
3. Request a dynamic 5-question multiple-choice quiz based on the uploaded content.

4. Interact with custom radio button selectors to answer the quiz, submit responses, and view detailed diagnostics explaining *why* wrong answers were incorrect.

Future Enhancements

- Voice-driven interactive Q&A (converting student voice to text and read-aloud tutoring replies).
- Multi-document indexing (allowing students to upload entire folders of notes).

2. Difficulty Estimation

- **Level:** Intermediate
- **Why:** Building a basic chatbot is a beginner task. However, this project is classified as *Intermediate* because:
  1. **RAG Pipeline Implementation:** Interns must coordinate document token parsing, text splitting thresholds, vector generation, and query retrieval using ChromaDB or FAISS.
  2. **Structured Outputs:** Generating interactive quizzes requires forcing the LLM to output valid JSON matching a strict schema (questions, options, correct answers). Parsing this output, handling malformed API payloads, and rendering stateful forms in Streamlit requires careful programming.

3. Skills Required

Programming

- **Python:** Core backend language.
- **Streamlit:** UI layout, sidebar files upload, chat components, and state management (preserving conversation history and active quiz data).
- **SQLite:** Recording quiz scores, user statistics, and logging processed files.

AI/ML

- **Retrieval-Augmented Generation (RAG):** Context injection, search thresholding, and prompt styling.
- **Vector Embeddings:** Generating and querying mathematical text representations.
- **Vector Databases:** Managing collection indexes, reading, writing, and similarity scoring.
- **Prompt Engineering:** Structuring system prompts, using few-shot examples, and demanding structured JSON outputs.

Software Engineering

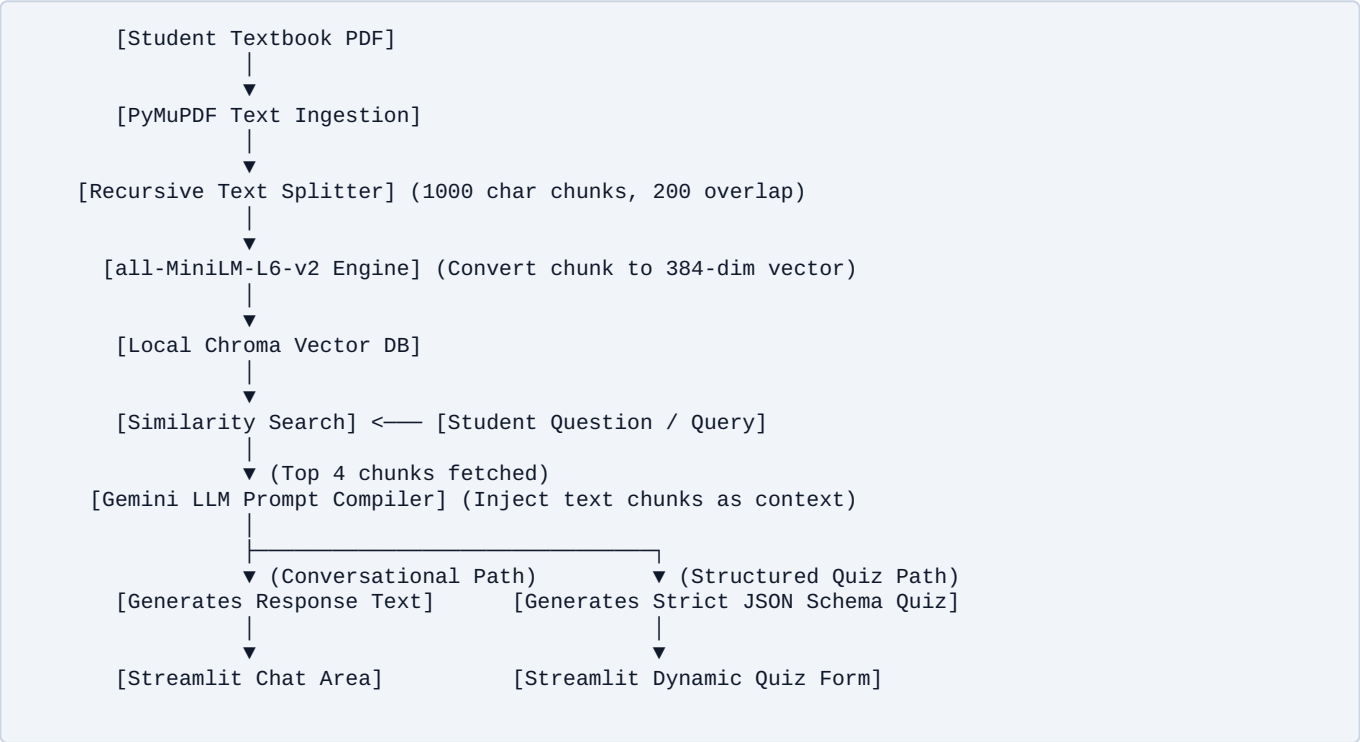
- **Git & GitHub:** Repository organization, regular commits, and README creation.
- **Document Processing:** Parsing PDF tables, paragraphs, and raw text layout using PyMuPDF.
- **API Integration:** Securing credentials using environmental variables (`.env`) and integrating LLM client SDKs.

4. Recommended Tech Stack

Component	Technology	Why
Frontend UI	**Streamlit**	Built-in chat widgets (`st.chat_message` , `st.chat_input`) and dynamic forms (`st.form`) allow creating modern AI interfaces quickly.
Document Parser	**PyMuPDF (fitz)**	One of the fastest, most reliable PDF parsing libraries in Python. Extracts metadata (like page numbers) along with raw text.
Vector DB	**ChromaDB (Local/Embedded)**	An open-source, developer-friendly vector database. Runs completely locally in-memory or persists in a local project folder with zero setup.
Embeddings Model	**Hugging Face (`all-MiniLM-L6-v2`)**	A highly popular, lightweight embedding model. Converts sentences to 384-dimensional dense vectors locally.
LLM Core API	**Google Gemini Developer API (`gemini-2.5-flash`)**	Generous free tier, extremely fast response latency, and supports native JSON Schema output configurations.
RAG Orchestrator	**LangChain**	Provides built-in text splitters (`RecursiveCharacterTextSplitter`) and vector loaders to avoid writing vector search mathematics from scratch.

5. System Architecture

Text-Based Architecture Flowchart



6. Development Modules

- **Module 1: Document Loader & Parser**
  Purpose: Process uploaded PDF file streams, extract text per page, filter headers, and log metadata (file name, page counts).
  Inputs: PDF byte buffer. | *Outputs:* Clean text dictionaries mapped by page numbers.
  Dependencies: `pymupdf`
- **Module 2: Embedding & Indexing Engine**
  Purpose: Chunk text inputs, generate semantic embeddings, and write them into the vector database.
  Inputs: Clean text dicts. | *Outputs:* Chroma DB collections written to disk.
  Dependencies: `chromadb`, `sentence-transformers`, `langchain-text-splitters`
- **Module 3: Similarity Retriever**
  Purpose: Perform cosine similarity search on the vector DB collection using user query parameters.
  Inputs: Student text query. | *Outputs:* Top K matching text chunks with page references.
  Dependencies: `chromadb`
- **Module 4: Prompt Constructor & LLM Controller**
  Purpose: Compile retriever context and user prompts into a system message and fetch response text from Gemini.
  Inputs: User query, matching text chunks. | *Outputs:* AI Tutoring response strings.
  Dependencies: `google-generativeai`

- **Module 5: Structured Quiz Generator**
  Purpose: Command Gemini to output a multiple-choice quiz derived from the document context matching a strict JSON schema.
  Inputs: Chapter context chunks. | *Outputs:* Valid JSON arrays of quiz questions, choices, answers, and explanations.
  Dependencies: `google-generativeai`, `pydantic` (for validation)
- **Module 6: Streamlit Tutoring Dashboard**
  Purpose: Provide sidebar file uploads, coordinate chatbot chat histories, render quizzes as stateful forms, and display performance scores.
  Inputs: Student selections, text commands. | *Outputs:* Dynamic screen updates, chat responses, and diagnostic metrics cards.
  Dependencies: `streamlit`

7. What Should Be Built vs Reused

Build Yourself	Reuse Existing
Streamlit Session State management	PDF text reader libraries (`pymupdf`)
Structured Pydantic validation models	Vector database indices (`ChromaDB`)
Context-injected LLM Prompt Templates	Pre-trained Embedding Models (`all-MiniLM-L6-v2`)
Custom quiz score diagnostic logic	Gemini SDK client libraries
Local database schemas (logs & scores in SQLite)	Streamlit chat message UI templates

8. Pre-trained Models and APIs

- `all-MiniLM-L6-v2` **(Sentence Transformers):** Free, local embedding generator. Excellent balance of speed, memory usage, and retrieval accuracy.
- **Google Gemini API (** `gemini-2.5-flash` **):** Used to power both the dialogic Q&A responses and the structured quiz creation.

9. Existing Open Source Projects to Study

- **LangChain RAG Quickstart Repository:** Review patterns of document ingestion, vector retrieval, and history injection.
- **Streamlit App Gallery (Generative AI section):** Study how multi-page layouts and conversational chat containers are built.
- **What NOT to copy:** Large enterprise RAG tools (like Dify or Flowise) which run complex Docker compose networks. Keep the implementation self-contained within Streamlit to ensure the student can run and modify all code directly.

10. Data Sources

- **OpenStax Academic Textbooks:** Free, high-quality, open-licensed college textbooks (Biology, Chemistry, College Physics, US History) in PDF format. Search keywords: `openstax textbook pdf downloads`. Cost: Free.
- **Synthetic Lecture Notes:** Custom markdown documents representing mock course outlines for testing.

11. Research Papers for Reference

Title	Year	Summary	Why Read It
Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	2020	Foundational paper introducing the concept of augmenting LLM generators with retrieved document contexts.	Crucial for understanding why RAG reduces LLM hallucinations.
Active Learning and Personalized Tutoring Systems	2015	Explains pedagogical theories behind testing student understanding with feedback loops.	Helps design better tutor system prompts.

12. Self-Learning Resources

Official Documentation

- **LangChain Text Splitters:** LangChain Docs - Essential for setting chunk sizes and overlap rules.
- **ChromaDB Get Started Guide:** Chroma Docs - Essential for understanding vector database collection management.

YouTube Crash Courses

- **RAG Pipeline Tutorial in Python:** *RAG from Scratch* (by LangChain or freeCodeCamp).
- **Gemini API Structured Output Guide:** search: "Gemini API JSON Mode Python".

13. Industry Standard Folder Structure

```
eduguide_tutor/
│
├── config/
│   └── .env.example        # Template for GEMINI_API_KEY
│
├── database/
│   ├── __init__.py
│   ├── db_manager.py       # SQLite connection to log quiz scores and file names
│   └── student_logs.db     # SQLite DB file (ignored in .gitignore)
│
├── storage/
│   └── chromadb/           # Chroma Vector database files directory (ignored in .gitignore)
│
├── app/
│   ├── __init__.py
│   ├── main.py             # Streamlit application main layout
│   ├── document_handler.py # PDF reader & text chunker functions
│   ├── vector_handler.py   # Embedding generator and Chroma db interface
│   ├── rag_engine.py       # Similarity retriever & Gemini LLM caller
│   └── quiz_engine.py      # Quiz generation parser & score evaluator
│
├── tests/
│   └── test_rag.py         # Unit tests checking text splitting and retrieval metrics
│
├── requirements.txt
├── .gitignore
├── README.md
└── run.py                  # Entry script launching streamlit (`streamlit run app/main.py`)
```

14. GitHub Milestones

1. **Milestone 1: Repository Setup & PDF Parser**
   Initialize directory structure, install dependencies, and build a Python script using PyMuPDF that extracts text from a PDF and prints it.
2. **Milestone 2: Chunker & Local Vector Database**
   Configure LangChain's text splitter, download embedding models, write vectors, and initialize a local ChromaDB collection.
3. **Milestone 3: Basic Retrieval Engine**
   Implement similarity searches. Verify that querying a topic returns the exact page/paragraph chunks containing the relevant information.
4. **Milestone 4: Contextual Tutoring (RAG Q&A)****
   Link vector retrieval to Gemini API. Design system prompts that instruct Gemini to answer queries *only* using the provided context, citing pages.
5. **Milestone 5: Structured Quiz Engine**
   Configure Gemini API to generate multiple-choice quizzes using JSON Mode and Pydantic schemas. Write validation checks.

6. **Milestone 6: Streamlit UI Implementation**
   Build the UI: PDF uploader sidebar, conversation container, and interactive quiz page using Streamlit session state variables.
7. **Milestone 7: Unit Testing & Performance Tuning**
   Add pytest cases, cache functions to optimize performance, write the `README.md`, and record the demo video.

15. MVP Planning

Must Have

- **PDF file upload widget in Streamlit sidebar.**
- **PDF parsed text chunking (1000 char size, 200 overlap) and Chroma DB.**
- **RAG Q&A chat interface with page citations.**
- **AI Quiz generator producing 5 MCQs.**
- **Stateful quiz interface with evaluation.**

Nice to Have

- **Dark/Light mode theme configurations.**
- **SQLite dashboard panel showing historical quiz attempt scores.**
- **Export feature: Download generated quizzes as study guides.**

Future Scope: Adaptive difficulty based on performance history; speech integration for voice feedback loops.

16. 15-Day Curriculum

Day	Phase	Topics & Learning Goals	What to Build & Expected Deliverable	Time
Day 1	Phase 1	Project setup, Venv, dependencies, Git.	Set up folders, Git tracker, active venv.	4 hrs
Day 2	Phase 1	PyMuPDF structures, page text arrays.	Build `document_handler.py` text parser.	5 hrs
Day 3	Phase 1	Text chunking, token limits, boundary limits.	Write recursive character text splitters.	5 hrs
Day 4	Phase 2	Vector embeddings math, similarity scoring.	Write embedding generation script.	6 hrs
Day 5	Phase 2	ChromaDB vector indexing, local storage schemas.	Develop index manager interface.	6 hrs
Day 6	Phase 2	Semantic similarity retrieval mechanics.	Implement search operations returning top 3 segments.	6 hrs
Day 7	Phase 2	Prompt design, context injection.	Connect Gemini API to RAG pipeline with references.	6 hrs
Day 8	Phase 2	Structured JSON generation, Pydantic constraints.	Build quiz generator engine using JSON mode.	6 hrs
Day 9	Phase 2	SQLite session logs, metrics tables.	Build database helper tracking student scores.	5 hrs
Day 10	Phase 3	Streamlit chat UI container architectures.	Draft chatbot interface with message bubbles.	6 hrs
Day 11	Phase 3	Stateful web form forms, radio validation states.	Build interactive multi-choice form evaluating inputs.	7 hrs
Day 12	Phase 3	Performance profiling, memory caching.	Add caching decorators to loaders.	6 hrs
Day 13	Phase 3	Pytest testing suites, API mock tools.	Develop data calculation tests under `tests/`.	6 hrs
Day 14	Phase 4	Cloud deployments, showcase guidelines.	Deploy to Streamlit cloud; record 3-min video.	4 hrs
Day 15	Phase 4	Slide presentation design templates.	Design summary milestones slide decks.	6 hrs

17. Daily Output Expectations & Developer Code Guides

Critical Developer Guide: Fetching a Structured Quiz from Gemini

```
import os
import json
from pydantic import BaseModel, Field
from typing import List
import google.generativeai as genai
from dotenv import load_dotenv

load_dotenv()

# Step 1: Define the strict data structure of the Quiz
class Question(BaseModel):
    question_text: str = Field(description="The question prompt testing textbook
understanding")
    options: List[str] = Field(description="Exactly 4 multiple choice options")
    correct_option_index: int = Field(description="Index (0 to 3) of the correct answer
option")
    explanation: str = Field(description="Pedagogical explanation why this answer is
correct and others are wrong")

class Quiz(BaseModel):
    quiz_title: str
    questions: List[Question]

def generate_quiz_from_context(text_context: str) -> Quiz:
    genai.configure(api_key=os.environ["GEMINI_API_KEY"])
    model = genai.GenerativeModel('gemini-2.5-flash')

    prompt = f"""
    You are an expert tutor. Generate a 5-question multiple choice quiz testing the
student's understanding
    of the following educational context:
    ---
    {text_context}
    ---
    Generate a quiz adhering to the requested JSON structure.
    """

    response = model.generate_content(
        prompt,
        generation_config=genai.GenerationConfig(
            response_mime_type="application/json",
            response_schema=Quiz
        )
    )
```

```
    quiz_data = json.loads(response.text)
    return Quiz(**quiz_data)
```

18. Final Deliverables

1. Source Code: Complete Python project directories including app pipelines.
2. GitHub Repository: Clean history of version control commits.
3. README.md: Explaining project context, setup instructions, and database details.
4. Local Database: Schema details and sample logged data.
5. Live Dashboard Link: Deployed Streamlit Cloud URL.
6. Project Report: Standard PDF detailing findings, cohort interpretations, and ML evaluation metrics.
7. Demo Video: A 3-minute video showing file uploading, retention heatmaps, customer filters, and churn classification.
8. Presentation Slides: Summary slides.

19. Evaluation Rubric (100 Marks)

Criteria	Marks	Details
Working Features	20	PDF parsing runs, RAG chat answers citation metrics, and quiz scoring processes inputs.
Code Quality	15	Code layouts, PEP 8 styling, cache configurations, and exception handlers.
AI Implementation	15	Chunking strategy efficacy, retrieval similarity accuracy, and JSON model validation.
Documentation	10	Detailed README.md, clean setup instructions, and PDF report.
Git Usage	10	Descriptive commits list, branch management, and repository hygiene.
UI/UX Design	10	Modern widgets layout, stateful forms, and clean chat bubbled interfaces.
Innovation & Extensions	10	Sidebar metrics graphs, adaptive quiz logic, or custom theme templates.
Testing	5	Pytest suites verifying the pipeline's computational logic.
Presentation & Demo	5	Video recording and slides explaining project value and findings.